Choreographer

Interface Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring. 5 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-Choreographer-go.

Choreographer - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/Choreographer. Entry point: ./cmd/choreographer. Module: arrowhead/ah5-support-go/choreographer. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8457 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8457"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Choreographer	Plan DAG validation, executor registration, synchronous orchestration, persistent steps/callbacks/worklog	Async result callback (explicit 501), complete dependency negotiation, optimal executor selection, session notification, remote abort propagation, inter-cloud sessions

Detailed implementation evidence

1 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-Choreographer-rust.

Choreographer (Rust)

Arrowhead 5.2.2 Choreographer support system. Entry point src/main.rs is built as binary choreographer by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/choreographer.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/Choreographer, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin choreographer -- <config.json>

Listening port

Default 127.0.0.1:8457 (Arrowhead default, role choreographer). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Choreographer	Plan/executor CRUD, single local action/step starts, aborts and completion callbacks	Dependencies, parallel/multi-action execution, quantities, optimal selection, auto discovery, notifications, async orchestration, exact executor DTO parity

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native

snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring/documentation/v5.2.2/Choreographer-v5.2.2-choreographerExecutor-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring/documentation/v5.2.2/Choreographer-v5.2.2-choreographerManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring/documentation/v5.2.2/Choreographer-v5.2.2-choreographerSession-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring/documentation/v5.2.2/Choreographer-v5.2.2-generalManagement-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-Choreographer-java-spring/documentation/v5.2.2/Choreographer-v5.2.2-sysMonitor-AMQP-IDD.tex

- Go: Eclipse-Arrowhead-go/ah5-Choreographer-go/documentation/Choreographer/Choreographer-HTTPS-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Choreographer/Choreographer-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

choreographerExecutor HTTP/TLS/JSON

Interface Design Description

Service ID: *"choreographerExecutor"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `choreographerExecutor` service of the Choreographer system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation register . . . 4
2.2 operation unregister . . . 4
2.3 operation notify-step-done . . . 5

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct ExecutedStepResult . . . 6
3.3 struct ExecutorRegistrationRequest . . . 6
3.4 struct ExecutorResponse . . . 6
3.5 struct ExecutorServiceDefinitionResponse . . . 7
3.6 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `choreographerExecutor` service interface of the Choreographer system, which provides registration of executor systems and reporting of executed plan steps.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `choreographerExecutor` service interface

This document provides the Interface Design Description IDD to the *choreographerExecutor – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `choreographerExecutor` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `choreographer`, port 8457 (`server.port`; the standalone deployment registers 8454) and base path `/choreographer/executor`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`register`	POST	`/register`
`unregister`	DELETE	`/unregister`
`notify-step-done`	POST	`/notify-step-done`

2 Service Operations

The interfaces of the `choreographerExecutor` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `choreographerExecutor` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /choreographer/executor/register

Operation: register
Input: ExecutorRegistrationRequest
Output: ExecutorResponse

Registers the requester as executor for a service definition (version window).

A successful call returns `201 Created` with the output in the body.

```
POST /choreographer/executor/register HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "address": "192.168.0.10",
  "port": 8080,
  "basePath": "/executor",
  "serviceDefinitionName": "temperatureReading",
  "minVersion": 1,
  "maxVersion": 1
}
```

Listing 1: An register invocation.

```
{
  "id": 42,
  "name": "TemperatureSensor",
  "address": "192.168.0.10",
  "port": 8080,
  "basePath": "/executor",
  "serviceDefinitions": [
    {
      "serviceDefinitionName": "temperatureReading",
      "minVersion": 1,
      "maxVersion": 1
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 2: An register response.

2.2 DELETE /choreographer/executor/unregister

Operation: unregister

Removes the requester's executor registration.
A successful call returns `200 OK`.

```
DELETE /choreographer/executor/unregister HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An unregister invocation.

2.3 POST /choreographer/executor/notify-step-done

Operation: notify-step-done
Input: ExecutedStepResult

Reports the result of a session step executed by the requester.
A successful call returns `200 OK`.

```
POST /choreographer/executor/notify-step-done HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "sessionId": 1,
  "sessionStepId": 1,
  "status": "SUCCESS",
  "message": "1 local result: TemperatureSensor"
}
```

Listing 4: An notify-step-done invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.6, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct ExecutedStepResult

Result of an executed step, reported by the executor.

Object Field	Value Type	Description
"sessionId"	Number	Identifier of the session.
"sessionStepId"	Number	Identifier of the executed session step.
"status"	ExecutedStepStatus	Result: `SUCCESS`, `ERROR`, `FATAL_ERROR` or `ABORTED`.
"message"	String	Result message.
"exception"	String	Error details, if any.

3.3 struct ExecutorRegistrationRequest

Registration of the requesting executor.

Object Field	Value Type	Description
"address"	String	Address of the executor; must be one of its ServiceRegistry addresses.
"port"	Number	Port of the executor interface.
"basePath"	String	Base path of the executor interface.
"serviceDefinitionName"	String	Service definition the executor can execute steps for.
"minVersion"	Number	Lowest accepted major version.
"maxVersion"	Number	Highest accepted major version.

3.4 struct ExecutorResponse

A registered executor.

Object Field	Value Type	Description
"id"	Number	Identifier of the executor.
"name"	String	Executor system.
"address"	String	Address of the executor interface.
"port"	Number	Port of the executor interface.
"basePath"	String	Base path of the executor interface.
"serviceDefinitions"	Array<ExecutorServiceDefinitionResponse>	Service definitions the executor serves.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.5 struct ExecutorServiceDefinitionResponse

A service definition an executor serves.

Object Field	Value Type	Description
"serviceDefinitionName"	String	Service definition name, camelCase (e.g. `temperatureReading`).
"minVersion"	Number	Lowest accepted major version.
"maxVersion"	Number	Highest accepted major version.

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
ExecutedStepStatus	One of `SUCCESS`, `ERROR`, `FATAL_ERROR`, `ABORTED` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “choreographerExecutor – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerExecutor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

choreographerManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"choreographerManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `choreographerManagement` service of the Choreographer system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query-plans . . . 4
2.2 operation get-plan . . . 5
2.3 operation create-plan . . . 5
2.4 operation remove-plan . . . 7
2.5 operation check-plan . . . 7
2.6 operation start-sessions . . . 7
2.7 operation abort-session . . . 8
2.8 operation query-sessions . . . 9
2.9 operation query-session-steps . . . 9
2.10 operation query-worklog . . . 10
2.11 operation query-executors . . . 10
2.12 operation get-executor . . . 11
2.13 operation create-executor . . . 12
2.14 operation remove-executor . . . 12

3 Data Models **14**
3.1 struct ActionRequest . . . 14
3.2 struct ActionResponse . . . 14
3.3 struct CheckPlanResponse . . . 14
3.4 struct ErrorMessage . . . 14
3.5 struct ExecutorCreateRequest . . . 14
3.6 struct ExecutorListResponse . . . 15
3.7 struct ExecutorResponse . . . 15
3.8 struct ExecutorServiceDefinitionResponse . . . 15
3.9 struct MetadataRequirement . . . 15
3.10 struct OrchestrationServiceRequirement . . . 16
3.11 struct PlanListResponse . . . 16
3.12 struct PlanRequest . . . 16
3.13 struct PlanResponse . . . 16
3.14 struct RunPlanListRequest . . . 17
3.15 struct RunPlanListResponse . . . 17
3.16 struct RunPlanRequest . . . 17
3.17 struct RunPlanResponse . . . 17
3.18 struct SessionListResponse . . . 17
3.19 struct SessionResponse . . . 18
3.20 struct SessionStepListResponse . . . 18
3.21 struct SessionStepResponse . . . 18
3.22 struct StepRequest . . . 19
3.23 struct StepResponse . . . 19
3.24 struct WorklogListResponse . . . 19
3.25 struct WorklogResponse . . . 20
3.26 Primitives . . . 20

4 References **21**

5 Revision History **22**
5.1 Amendments . . . 22
5.2 Quality Assurance . . . 22

1 Overview

This document describes the `choreographerManagement` service interface of the Choreographer system, which provides operator management of plans, sessions, session steps, the worklog and executors.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `choreographerManagement` service interface

This document provides the Interface Design Description IDD to the *choreographerManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `choreographerManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `choreographer`, port 8457 (`server.port`; the standalone deployment registers 8454) and base path `/choreographer/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query-plans`	GET	`/plans`
`get-plan`	GET	`/plans/{id}`
`create-plan`	POST	`/plans`
`remove-plan`	DELETE	`/plans/{id}`
`check-plan`	GET	`/plans/{id}/check`
`start-sessions`	POST	`/sessions/start`
`abort-session`	DELETE	`/sessions/{id}`
`query-sessions`	GET	`/sessions`
`query-session-steps`	GET	`/session-steps`
`query-worklog`	GET	`/worklog`
`query-executors`	GET	`/executors`
`get-executor`	GET	`/executors/{id}`
`create-executor`	POST	`/executors`
`remove-executor`	DELETE	`/executors/{id}`

2 Service Operations

The interfaces of the `choreographerManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `choreographerManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 GET /choreographer/mgmt/plans?page={page}&size={size}&direction={direction}&sortField={sortField}

Operation: query-plans
Output: PlanListResponse

Returns a page of plans.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sortField* (String, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/plans?page=<page>&size=<size>&direction=<direction>&sortField=<sortField>
    HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An query-plans invocation.


```
{
  "entries": [
    {
      "id": 42,
      "name": "TemperatureSensor",
      "firstActionName": "string",
      "actions": [
        {
          "id": 42,
          "name": "TemperatureSensor",
          "firstAction": true,
          "nextActionName": "string",
          "firstStepNames": [
            "string"
          ],
          "steps": [
            {}
          ],
          "createdAt": "2026-10-22T12:00:00Z",
          "updatedAt": "2026-10-22T12:00:00Z"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
```

```
}
```

Listing 2: An query-plans response.

2.2 GET /choreographer/mgmt/plans/*{id}*

Operation: get-plan
Input: String
Output: PlanResponse

Returns a plan.

Path parameter: *id*.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/plans/<id> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-plan invocation.

```
{
  "id": 42,
  "name": "TemperatureSensor",
  "firstActionName": "string",
  "actions": [
    {
      "id": 42,
      "name": "TemperatureSensor",
      "firstAction": true,
      "nextActionName": "string",
      "firstStepNames": [
        "string"
      ],
      "steps": [
        {
          "id": 42,
          "name": "TemperatureSensor",
          "serviceDefinition": "temperatureReading",
          "minVersion": 1,
          "maxVersion": 1,
          "quantity": 1,
          "localCloudOnly": true,
          "nextStepNames": [
            "string"
          ],
          "createdAt": "2026-10-22T12:00:00Z",
          "updatedAt": "2026-10-22T12:00:00Z"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 4: An get-plan response.

2.3 POST /choreographer/mgmt/plans

Operation: create-plan
Input: PlanRequest
Output: PlanResponse

Validates and stores a plan (cycles, reachability, transitive reduction).
A successful call returns `201 Created` with the output in the body.

```
POST /choreographer/mgmt/plans HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "name": "TemperatureSensor",
  "firstActionName": "string",
  "actions": [
    {
      "name": "TemperatureSensor",
      "nextActionName": "string",
      "firstStepNames": [
        "string"
      ],
      "steps": [
        {
          "name": "TemperatureSensor",
          "quantity": 1,
          "localCloudOnly": true,
          "nextStepNames": [
            "string"
          ]
        }
      ]
    }
  ]
}
```

Listing 5: An create-plan invocation.

```
{
  "id": 42,
  "name": "TemperatureSensor",
  "firstActionName": "string",
  "actions": [
    {
      "id": 42,
      "name": "TemperatureSensor",
      "firstAction": true,
      "nextActionName": "string",
      "firstStepNames": [
        "string"
      ],
      "steps": [
        {
          "id": 42,
          "name": "TemperatureSensor",
          "serviceDefinition": "temperatureReading",
          "minVersion": 1,
          "maxVersion": 1,
          "quantity": 1,
          "localCloudOnly": true,
          "nextStepNames": [
            "string"
          ],
          "createdAt": "2026-10-22T12:00:00Z",
```

```
          "updatedAt": "2026-10-22T12:00:00Z"
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 6: An create-plan response.

2.4 DELETE /choreographer/mgmt/plans/*{id}*

Operation: remove-plan
Input: String

Removes a plan.
Path parameter: *id*.
A successful call returns `200 OK`.

```
DELETE /choreographer/mgmt/plans/<id> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 7: An remove-plan invocation.

2.5 GET /choreographer/mgmt/plans/*{id}*/check?allowInterCloud=*{allowInterCloud}*

Operation: check-plan
Input: String
Output: CheckPlanResponse

Checks whether providers and executors exist for every step.
Path parameter: *id*.
Query parameters: *allowInterCloud* (boolean, optional, default `false`). List values are repeated or comma-separated.
A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/plans/<id>/check?allowInterCloud=<allowInterCloud> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 8: An check-plan invocation.

```
{
  "planId": 1,
  "errorMessages": [
    "string"
  ],
  "needInterCloud": true
}
```

Listing 9: An check-plan response.

2.6 POST /choreographer/mgmt/sessions/start

Operation: start-sessions
Input: RunPlanListRequest
Output: RunPlanListResponse

Starts sessions of plans by identifier.
A successful call returns `201 Created` with the output in the body.

```
POST /choreographer/mgmt/sessions/start HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "sessions": [
    {
      "planId": 1,
      "quantity": 1,
      "allowInterCloud": true,
      "chooseOptimalExecutor": true,
      "notifyProtocol": "HTTPS",
      "notifyAddress": "192.168.0.20",
      "notifyPort": 8080,
      "notifyPath": "/notify"
    }
  ]
}
```

Listing 10: An start-sessions invocation.

```
{
  "entries": [
    {
      "planId": 1,
      "sessionId": 1,
      "quantity": 1,
      "status": "INITIATED",
      "errorMessages": [
        "string"
      ],
      "needInterCloud": true
    }
  ]
}
```

Listing 11: An start-sessions response.

2.7 DELETE /choreographer/mgmt/sessions/*{id}*

Operation: abort-session
Input: String

Aborts a session.
Path parameter: *id*.
A successful call returns `200 OK`.

```
DELETE /choreographer/mgmt/sessions/<id> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 12: An abort-session invocation.

2.8 GET /choreographer/mgmt/sessions?page={page}&size={size}&direction={direction}&sortField={sortField}&planId={planId}&status={status}

Operation: query-sessions
Output: SessionListResponse

Returns a page of sessions.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sortField* (String, optional); *planId* (Long, optional); *status* (String, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/sessions?page=<page>&size=<size>&direction=<direction>&sortField=<sortField
    >&planId=<planId>&status=<status> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 13: An query-sessions invocation.


```
{
  "entries": [
    {
      "id": 42,
      "planId": 1,
      "planName": "fillTank",
      "status": "INITIATED",
      "quantityDone": 1,
      "quantityGoal": 1,
      "executionNumber": 1,
      "owner": "TemperatureDisplay",
      "notifyUri": "https://display:8080/events",
      "startedAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```


Listing 14: An query-sessions response.

2.9 GET /choreographer/mgmt/session-steps?page={page}&size={size}&direction={direction}&sortField={sortField}&sessionId={sessionId}&status={status}

Operation: query-session-steps
Output: SessionStepListResponse

Returns a page of session steps.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sortField* (String, optional); *sessionId* (Long, optional); *status* (String, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/session-steps?page=<page>&size=<size>&direction=<direction>&sortField=<
    sortField>&sessionId=<sessionId>&status=<status> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 15: An query-session-steps invocation.

```
{
  "entries": [
    {
      "id": 42,
      "sessionId": 1,
      "stepId": 1,
      "planName": "fillTank",
      "actionName": "string",
      "stepName": "string",
      "executorName": "string",
      "status": "WAITING",
      "executionNumber": 1,
      "message": "1 local result: TemperatureSensor",
      "startedAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 16: An query-session-steps response.

2.10 GET /choreographer/mgmt/worklog?page={page}&size={size} &direction={direction}&sortField={sortField}&sessionId={sessionId} &planName={planName}&actionName={actionName}&stepName={stepName}

Operation: query-worklog
Output: WorklogListResponse

Returns a page of worklog entries.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sortField* (String, optional); *sessionId* (Long, optional); *planName* (String, optional); *actionName* (String, optional); *stepName* (String, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/worklog?page=<page>&size=<size>&direction=<direction>&sortField=<sortField>&
    sessionId=<sessionId>&planName=<planName>&actionName=<actionName>&stepName=<stepName> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 17: An query-worklog invocation.

```
{
  "entries": [
    {
      "id": 42,
      "entryDate": "2026-10-22T12:00:00Z",
      "planName": "fillTank",
      "actionName": "string",
      "stepName": "string",
      "sessionId": 1,
      "executionNumber": 1,
      "message": "1 local result: TemperatureSensor"
    }
  ],
  "count": 1
}
```

Listing 18: An query-worklog response.

2.11 GET /choreographer/mgmt/executors?page={page}&size={size}&direction={direction}&sortField={sortField}

Operation: query-executors
Output: ExecutorListResponse

Returns a page of executors.

Query parameters: *page* (Integer, optional); *size* (Integer, optional); *direction* (String, optional); *sortField* (String, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/executors?page=<page>&size=<size>&direction=<direction>&sortField=<sortField
    > HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 19: An query-executors invocation.

```
{
  "entries": [
    {
      "id": 42,
      "name": "TemperatureSensor",
      "address": "192.168.0.10",
      "port": 8080,
      "basePath": "/executor",
      "serviceDefinitions": [
        {
          "serviceDefinitionName": "temperatureReading",
          "minVersion": 1,
          "maxVersion": 1
        }
      ],
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z"
    }
  ],
  "count": 1
}
```

Listing 20: An query-executors response.

2.12 GET /choreographer/mgmt/executors/{id}

Operation: get-executor
Input: String
Output: ExecutorResponse

Returns an executor.

Path parameter: *id*.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/mgmt/executors/<id> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 21: An get-executor invocation.

```
{
  "id": 42,
```

```
  "name": "TemperatureSensor",
  "address": "192.168.0.10",
  "port": 8080,
  "basePath": "/executor",
  "serviceDefinitions": [
    {
      "serviceDefinitionName": "temperatureReading",
      "minVersion": 1,
      "maxVersion": 1
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 22: An get-executor response.

2.13 POST /choreographer/mgmt/executors

Operation: create-executor
Input: ExecutorCreateRequest
Output: ExecutorResponse

Registers an executor on behalf of a system.

A successful call returns `201 Created` with the output in the body.

```
POST /choreographer/mgmt/executors HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "systemName": "TemperatureSensor",
  "address": "192.168.0.10",
  "port": 8080,
  "basePath": "/executor",
  "serviceDefinitionName": "temperatureReading",
  "minVersion": 1,
  "maxVersion": 1
}
```

Listing 23: An create-executor invocation.

```
{
  "id": 42,
  "name": "TemperatureSensor",
  "address": "192.168.0.10",
  "port": 8080,
  "basePath": "/executor",
  "serviceDefinitions": [
    {
      "serviceDefinitionName": "temperatureReading",
      "minVersion": 1,
      "maxVersion": 1
    }
  ],
  "createdAt": "2026-10-22T12:00:00Z",
  "updatedAt": "2026-10-22T12:00:00Z"
}
```

Listing 24: An create-executor response.

2.14 DELETE /choreographer/mgmt/executors/*{id}*

Operation: remove-executor
Input: String

Removes an executor.
Path parameter: *id*.
A successful call returns `200 OK`.

```
DELETE /choreographer/mgmt/executors/<id> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 25: An remove-executor invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.26, which are used to represent things like hashes, identifiers and texts.

3.1 struct ActionRequest

An action of a plan.

Object Field	Value Type	Description
"name"	String	Name of the action within the plan.
"nextActionName"	String	Action executed after this one.
"firstStepNames"	Array<String>	Steps that start the action.
"steps"	Array<StepRequest>	The steps of the action.

3.2 struct ActionResponse

An action of a plan.

Object Field	Value Type	Description
"id"	Number	Identifier of the action.
"name"	String	Name of the action within the plan.
"firstAction"	Boolean	Whether this is the plan's first action.
"nextActionName"	String	Action executed after this one.
"firstStepNames"	Array<String>	Steps that start the action.
"steps"	Array<StepResponse>	The steps of the action.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.3 struct CheckPlanResponse

Result of checking a plan against the current cloud.

Object Field	Value Type	Description
"planId"	Number	The checked plan.
"errorMessages"	Array<String>	Problems found in the plan.
"needInterCloud"	Boolean	Whether the plan needs providers in other clouds.

3.4 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.5 struct ExecutorCreateRequest

An executor to create on behalf of a system.

Object Field	Value Type	Description
"systemName"	String	Executor system.
"address"	String	One network address (host name, IPv4, IPv6 or MAC address).
"port"	Number	Port of the executor interface.
"basePath"	String	Base path of the executor interface.
"serviceDefinitionName"	String	Service definition the executor can execute steps for.
"minVersion"	Number	Lowest accepted major version.
"maxVersion"	Number	Highest accepted major version.

3.6 struct ExecutorListResponse

List of executors.

Object Field	Value Type	Description
"entries"	Array<ExecutorResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct ExecutorResponse

A registered executor.

Object Field	Value Type	Description
"id"	Number	Identifier of the executor.
"name"	String	Executor system.
"address"	String	Address of the executor interface.
"port"	Number	Port of the executor interface.
"basePath"	String	Base path of the executor interface.
"serviceDefinitions"	Array<ExecutorServiceDefinitionResponse>	Service definitions the executor serves.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.8 struct ExecutorServiceDefinitionResponse

A service definition an executor serves.

Object Field	Value Type	Description
"serviceDefinitionName"	String	Service definition name, camelCase (e.g. `temperatureReading`).
"minVersion"	Number	Lowest accepted major version.
"maxVersion"	Number	Highest accepted major version.

3.9 struct MetadataRequirement

A JSON object of metadata requirements: each key is a metadata path (dot notation), each value the required value or an operator object such as `{"op": "LESS_THAN", "value": 10}`.

3.10 struct OrchestrationServiceRequirement

Requirements on the requested service.

Object Field	Value Type	Description
"serviceDefinition"	String	Service definition name.
"operations"	Array<String>	Service operations the consumer wants to use.
"versions"	Array<Version>	Accepted service versions.
"alivesAt"	DateTime	Only instances that are still registered (not expired) at this UTC time are matched.
"metadata Requirements"	Array<MetadataRequirement>	Requirements on the service instance metadata; each map is one alternative.
"interfaceTemplate Names"	Array<String>	Accepted interface templates, e.g. `generic_https`.
"interfaceAddressTypes"	Array<String>	Address types the provider interface must offer.
"interfaceProperty Requirements"	Array<MetadataRequirement>	Requirements on the interface properties; each map is one alternative.
"securityPolicies"	Array<String>	Accepted interface security policies.
"preferredProviders"	Array<String>	Provider systems to prefer.

3.11 struct PlanListResponse

List of plans.

Object Field	Value Type	Description
"entries"	Array<PlanResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.12 struct PlanRequest

A plan to create.

Object Field	Value Type	Description
"name"	String	Unique plan name.
"firstActionName"	String	Action the plan starts with.
"actions"	Array<ActionRequest>	The actions of the plan.

3.13 struct PlanResponse

A stored plan.

Object Field	Value Type	Description
"id"	Number	Identifier of the plan.
"name"	String	Unique plan name.
"firstActionName"	String	Action the plan starts with.
"actions"	Array<ActionResponse>	The actions of the plan.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.14 struct RunPlanListRequest

Sessions to start by plan identifier.

Object Field	Value Type	Description
"sessions"	Array<RunPlanRequest>	Sessions to start or return.

3.15 struct RunPlanListResponse

Started sessions.

Object Field	Value Type	Description
"entries"	Array<RunPlanResponse>	Matching records of the current page.

3.16 struct RunPlanRequest

Start of a session of a plan.

Object Field	Value Type	Description
"planId"	Number	Identifier of the plan.
"quantity"	Number	Number of times the plan (or step) is executed.
"allowInterCloud"	Boolean	Allow inter-cloud orchestration for the steps.
"chooseOptimal Executor"	Boolean	Rank the local executors and choose the best one.
"notifyProtocol"	String	Protocol of session notifications, e.g. `HTTPS`.
"notifyAddress"	String	Address of session notifications.
"notifyPort"	Number	Port of session notifications.
"notifyPath"	String	Path of session notifications.

3.17 struct RunPlanResponse

A started (or refused) session.

Object Field	Value Type	Description
"planId"	Number	Identifier of the plan.
"sessionId"	Number	Identifier of the session.
"quantity"	Number	Requested executions.
"status"	SessionStatus	Status of the session.
"errorMessages"	Array<String>	Problems found in the plan.
"needInterCloud"	Boolean	Whether the plan needs providers in other clouds.

3.18 struct SessionListResponse

List of sessions.

Object Field	Value Type	Description
"entries"	Array<SessionResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.19 struct SessionResponse

A session.

Object Field	Value Type	Description
"id"	Number	Identifier of the session.
"planId"	Number	Identifier of the plan.
"planName"	String	Name of the plan.
"status"	SessionStatus	Session status.
"quantityDone"	Number	Executions finished so far.
"quantityGoal"	Number	Executions requested.
"executionNumber"	Number	Number of the current execution.
"owner"	String	System that started the session.
"notifyUri"	String	Where session notifications are sent.
"startedAt"	DateTime	UTC time when the job started.
"updatedAt"	DateTime	UTC time of the last change.

3.20 struct SessionStepListResponse

List of session steps.

Object Field	Value Type	Description
"entries"	Array<SessionStepResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.21 struct SessionStepResponse

A step execution of a session.

Object Field	Value Type	Description
"id"	Number	Identifier of the session step.
"sessionId"	Number	Identifier of the session.
"stepId"	Number	Identifier of the plan step.
"planName"	String	Name of the plan.
"actionName"	String	Name of the action.
"stepName"	String	Name of the step.
"executorName"	String	Executor system running the step.
"status"	SessionStepStatus	Status of the session step.
"executionNumber"	Number	Number of the current execution.
"message"	String	Text of the log entry or status message of the job.
"startedAt"	DateTime	UTC time when the job started.
"updatedAt"	DateTime	UTC time of the last change.

3.22 struct StepRequest

A step of an action.

Object Field	Value Type	Description
"name"	String	Name of the step within the action.
"quantity"	Number	Number of times the plan (or step) is executed.
"serviceRequirement"	OrchestrationServiceRequirement	Orchestration requirement the executor's provider must meet.
"localCloudOnly"	Boolean	Use only providers of the local cloud.
"staticParameters"	Object<String>	Parameters passed to the executor unchanged.
"nextStepNames"	Array<String>	Steps that start when this one has finished.

3.23 struct StepResponse

A step of an action.

Object Field	Value Type	Description
"id"	Number	Identifier of the step.
"name"	String	Name of the step within the action.
"serviceDefinition"	String	Service definition of the step's requirement.
"minVersion"	Number	Lowest major version derived from `versions`.
"maxVersion"	Number	Highest major version derived from `versions`.
"quantity"	Number	Number of times the plan (or step) is executed.
"serviceRequirement"	OrchestrationServiceRequirement	Orchestration requirement the executor's provider must meet.
"localCloudOnly"	Boolean	Use only providers of the local cloud.
"staticParameters"	Object<String>	Parameters passed to the executor unchanged.
"nextStepNames"	Array<String>	Steps that start when this one has finished.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.

3.24 struct WorklogListResponse

List of worklog entries.

Object Field	Value Type	Description
"entries"	Array<WorklogResponse>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.25 struct WorklogResponse

A worklog entry.

Object Field	Value Type	Description
"id"	Number	Identifier of the worklog entry.
"entryDate"	DateTime	UTC time of the log entry.
"planName"	String	Name of the plan.
"actionName"	String	Name of the action.
"stepName"	String	Name of the step.
"sessionId"	Number	Identifier of the session.
"executionNumber"	Number	Number of the current execution.
"message"	String	Worklog text.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.26 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
SessionStatus	One of `INITIATED`, `RUNNING`, `ABORTED`, `DONE` (JSON String).
SessionStepStatus	One of `WAITING`, `RUNNING`, `ABORTED`, `DONE` (JSON String).
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “choreographerManagement – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

choreographerSession HTTP/TLS/JSON

Interface Design Description

Service ID: *"choreographerSession"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `choreographerSession` service of the Choreographer system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation start-session . . . 4
2.2 operation abort-session . . . 5

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct RunPlanListResponse . . . 6
3.3 struct RunPlanResponse . . . 6
3.4 struct StartSessionListRequest . . . 6
3.5 struct StartSessionRequest . . . 6
3.6 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `choreographerSession` service interface of the Choreographer system, which provides starting and aborting sessions of workflow plans by their clients.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `choreographerSession` service interface

This document provides the Interface Design Description IDD to the *choreographerSession – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `choreographerSession` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `choreographer`, port 8457 (`server.port`; the standalone deployment registers 8454) and base path `/choreographer/session`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`start-session`	POST	`/start`
`abort-session`	DELETE	`/abort/{id}`

2 Service Operations

The interfaces of the `choreographerSession` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `choreographerSession` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /choreographer/session/start

Operation: start-session
Input: StartSessionListRequest
Output: RunPlanListResponse

Starts sessions of plans by name for the requester.

A successful call returns `201 Created` with the output in the body.

```
POST /choreographer/session/start HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "sessions": [
    {
      "planName": "fillTank",
      "quantity": 1,
      "allowInterCloud": true,
      "chooseOptimalExecutor": true,
      "notifyProtocol": "HTTPS",
      "notifyAddress": "192.168.0.20",
      "notifyPort": 8080,
      "notifyPath": "/notify"
    }
  ]
}
```

Listing 1: An start-session invocation.

```
{
  "entries": [
    {
      "planId": 1,
      "sessionId": 1,
      "quantity": 1,
      "status": "INITIATED",
      "errorMessages": [
        "string"
      ],
      "needInterCloud": true
    }
  ]
}
```

Listing 2: An start-session response.

2.2 DELETE /choreographer/session/abort/*{id}*

Operation: **abort-session**
Input: **String**

Aborts a session owned by the requester.
Path parameter: *id*.
A successful call returns `200 OK`.

```
DELETE /choreographer/session/abort/<id> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An abort-session invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.6, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct RunPlanListResponse

Started sessions.

Object Field	Value Type	Description
"entries"	Array<RunPlanResponse>	Matching records of the current page.

3.3 struct RunPlanResponse

A started (or refused) session.

Object Field	Value Type	Description
"planId"	Number	Identifier of the plan.
"sessionId"	Number	Identifier of the session.
"quantity"	Number	Requested executions.
"status"	SessionStatus	Status of the session.
"errorMessages"	Array<String>	Problems found in the plan.
"needInterCloud"	Boolean	Whether the plan needs providers in other clouds.

3.4 struct StartSessionListRequest

Sessions to start by plan name.

Object Field	Value Type	Description
"sessions"	Array<StartSessionRequest>	Sessions to start or return.

3.5 struct StartSessionRequest

Start of a session of a plan.

Object Field	Value Type	Description
"planName"	String	Name of the plan.
"quantity"	Number	Number of times the plan (or step) is executed.
"allowInterCloud"	Boolean	Allow inter-cloud orchestration for the steps.
"chooseOptimal Executor"	Boolean	Rank the local executors and choose the best one.
"notifyProtocol"	String	Protocol of session notifications, e.g. `HTTPS`.
"notifyAddress"	String	Address of session notifications.
"notifyPort"	Number	Port of session notifications.
"notifyPath"	String	Path of session notifications.

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
SessionStatus	One of `INITIATED`, `RUNNING`, `ABORTED`, `DONE` (JSON String).
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “choreographerSession – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-choreographerSession-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the Choreographer system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the Choreographer system, which provides general management of the Choreographer system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `choreographer`, port 8457 (`server.port`; the standalone deployment registers 8454) and base path `/choreographer/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /choreographer/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /choreographer/general/mgmt/logs HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /choreographer/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /choreographer/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: choreographer:8457
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the Choreographer system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation echo . . . 4
2.2 operation power . . . 4
2.3 operation status . . . 4

3 Data Models **6**
3.1 struct Observation . . . 6
3.2 Primitives . . . 6

4 References **7**

5 Revision History **8**
5.1 Amendments . . . 8
5.2 Quality Assurance . . . 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of Choreographer.
It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, Choreographer registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.Choreographer.Choreographer",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher is the `SysMonitorPublisher` of the common library, configured with `sysmonitor.broker.address`, `sysmonitor.broker.port`, `sysmonitor.local.cloud` (default: from the system certificate), `sysmonitor.publish.interval` and `sysmonitor.ssl.*` (default: the `server.ssl` stores). The publisher connects to the broker (default `amqp-broker`, port 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.Choreographer.Choreographer.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.Choreographer.Choreographer.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.Choreographer.Choreographer.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "Choreographer-echo-1792670400000-42",
  "providerSystem": "Choreographer",
  "serviceInstance": "Choreographer",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "Choreographer is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
}
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power`, `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (Choreographer),” Eclipse Arrowhead, SD 5.2.2, 2026, choreographer-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

HTTPS, mandatory mutual TLS, Arrowhead identity-token header and JSON request/response documents. General management is sysop/whitelist protected; enrollment endpoints use approved certificate fingerprints and profile/serial checks instead of ordinary identity tokens. Error responses contain errorMessage, errorCode, origin and timestamp. Exact DTO and status/error parity is not certified.

1.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/sessions	management
GET	/choreographer/mgmt/session-steps	management
GET	/choreographer/mgmt/worklog	management
GET	/choreographer/mgmt/executors	management
GET	/choreographer/mgmt/plans/{id}	management
DELETE	/choreographer/mgmt/plans/{id}	management
GET	/choreographer/mgmt/executors/{id}	management
DELETE	/choreographer/mgmt/executors/{id}	management
POST	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/plans/{id}/check	management
POST	/choreographer/mgmt/executors	management
POST	/choreographer/executor/register	identity
DELETE	/choreographer/executor/unregister	identity
POST	/choreographer/session/start	identity
POST	/choreographer/mgmt/sessions/start	management
DELETE	/choreographer/session/abort/{id}	identity
DELETE	/choreographer/mgmt/sessions/{id}	management
POST	/choreographer/executor/notify-step-done	identity
POST	/choreographer/orchestration-result/{correlationId}	identity
GET	/choreographer/monitor/echo	certificate
GET	/choreographer/general/mgmt/get-config	management
POST	/choreographer/general/mgmt/logs	management

1.2 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Local synchronous orchestration and executor start/callback flow are implemented. Async orchestration callbacks return 501. Java executor dependency negotiation, optimal selection, session notifications and remote abort propagation are incomplete.

1 Choreographer

1.1 System overview

Native Rust development profile for local Java source `Choreography/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Plan/executor CRUD, single local action/step starts, aborts and completion callbacks

1.5 Open release gates

Dependencies, parallel/multi-action execution, quantities, optimal selection, auto discovery, notifications, async orchestration, exact executor DTO parity. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `choreographerExecutor`, `choreographerManagement`, `choreographerSession`, `echo`, `generalManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/choreographer/general/mgmt/get-config	management
POST	/choreographer/general/mgmt/logs	management
GET	/choreographer/monitor/echo	certificate
POST	/choreographer/executor/register	identity
DELETE	/choreographer/executor/unregister	identity
POST	/choreographer/executor/notify-step-done	identity
POST	/choreographer/session/start	identity
DELETE	/choreographer/session/abort/{id}	identity
POST	/choreographer/orchestration-result/{correlationId}	identity / 501
GET	/choreographer/mgmt/plans	management
POST	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/plans/{id}	management
DELETE	/choreographer/mgmt/plans/{id}	management
GET	/choreographer/mgmt/plans/{id}/check	management
POST	/choreographer/mgmt/sessions/start	management
DELETE	/choreographer/mgmt/sessions/{id}	management
GET	/choreographer/mgmt/sessions	management
GET	/choreographer/mgmt/session-steps	management
GET	/choreographer/mgmt/worklog	management
GET	/choreographer/mgmt/executors	management
POST	/choreographer/mgmt/executors	management
GET	/choreographer/mgmt/executors/{id}	management
DELETE	/choreographer/mgmt/executors/{id}	management

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.